

Exercise 1: Containerizing a C++ Microservice using Docker

Objective

The objective of this exercise is to package a simple C++ microservice into a Docker container. The application is built, containerized, and executed using Docker in an Ubuntu-based cloud lab environment.

System Requirements

- Ubuntu-based Cloud Lab
- Docker installed and running
- No SSH access required
- Web-based terminal access

Project Directory Structure

```
cpp-docker/  
├── server.cpp  
└── Dockerfile
```

Step 1: Creating the C++ Microservice

A simple C++ HTTP server is created which listens on port 8080 and responds with a text message.

Screenshot Placeholder:

```
GNU nano 6.2 server.cpp
#include <iostream>
#include <cstring>
#include <unistd.h>
#include <arpa/inet.h>

int main() {
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);

    sockaddr_in address{};
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(8080);

    bind(server_fd, (struct sockaddr*)&address, sizeof(address));
    listen(server_fd, 3);

    std::cout << "Server running on port 8080..." << std::endl;

    while (true) {
        int client = accept(server_fd, nullptr, nullptr);

        const char* response =
            "HTTP/1.1 200 OK\n"
            "Content-Type: text/plain\n\n"
            "Hello from C++ Docker container!";

        send(client, response, strlen(response), 0);
        close(client);
    }
}
```

Step 2: Creating the Dockerfile

A Dockerfile is written to define the base image, install required dependencies, compile the C++ application, and run it inside a container.

Screenshot Placeholder:

```
student@student-virtual-machine: ~/25SUB4508_LSP/25SUB4508_56133/As... x student@student-virtual-machine: ~/25SUB4508_LSP/25SUB4508_56133/As... x
GNU nano 6.2 Dockerfile
FROM ubuntu:22.04

RUN apt-get update && apt-get install -y g++

WORKDIR /app

COPY server.cpp .

RUN g++ server.cpp -o server

EXPOSE 8080

CMD ["/server"]
```

Step 3: Building the Docker Image

The Docker image is built using the Docker CLI command. This process packages the application and its dependencies into a container image.

Screenshot Placeholder:

```
student@student-virtual-machine: ~/25SUB4508_LSP/25SUB4508_56133/Assignments/day35_CA_Software/Exercise_1$ cd cpp-docker
student@student-virtual-machine: ~/25SUB4508_LSP/25SUB4508_56133/Assignments/day35_CA_Software/Exercise_1/cpp-docker$ ll
total 16
drwxrwxr-x 2 student student 4096 Feb  5 09:36 ./
drwxrwxr-x 3 student student 4096 Feb  5 09:36 ../
-rw-rw-r-- 1 student student 158 Feb  5 09:36 Dockerfile
-rw-rw-r-- 1 student student 765 Feb  5 09:36 server.cpp
student@student-virtual-machine: ~/25SUB4508_LSP/25SUB4508_56133/Assignments/day35_CA_Software/Exercise_1/cpp-docker$ docker build -t cpp-microservice .
[+] Building 64.5s (10/10) FINISHED
=> [internal] load build definition from Dockerfile                                docker:default 0.0s
=> => transferring dockerfile: 197B                                             0.0s
=> [internal] load metadata for docker.io/library/ubuntu:22.04                 2.4s
=> [internal] load .dockerignore                                                 0.0s
=> => transferring context: 2B                                                  0.0s
=> CACHED [1/5] FROM docker.io/library/ubuntu:22.04@sha256:c7eb020043d8fc2ae0793fb35a37bff1cf33f156d4d4b12ccc7f3ef8706c38b1 0.0s
=> => resolve docker.io/library/ubuntu:22.04@sha256:c7eb020043d8fc2ae0793fb35a37bff1cf33f156d4d4b12ccc7f3ef8706c38b1 0.0s
=> [internal] load build context                                                0.0s
=> => transferring context: 804B                                               0.0s
=> [2/5] RUN apt-get update && apt-get install -y g++                          33.4s
=> [3/5] WORKDIR /app                                                         0.1s
=> [4/5] COPY server.cpp .                                                    0.0s
=> [5/5] RUN g++ server.cpp -o server                                          0.9s
=> => exporting to image                                                         27.5s
=> => exporting layers                                                         23.3s
=> => exporting manifest sha256:eda0ae421e8db042e1a0e2b87989261345982abdea4ed45c9d0267222c283a81 0.0s
=> => exporting config sha256:f73e878ea65c5b03f551cd9e1ae3d562b0c2766908b17a56a4f09f1fc56806b8 0.0s
=> => exporting attestation manifest sha256:831f51cb9618cadf4747d1a43b6bef0d17a60e7ef5707e0475f57a034ec9db24 0.0s
=> => exporting manifest list sha256:259af2fa30e9de00e59c1954e7a79a7fc959450fa6d593f6acb69c35808d2da9 0.0s
=> => naming to docker.io/library/cpp-microservice:latest                     0.0s
=> => unpacking to docker.io/library/cpp-microservice:latest                  4.2s
student@student-virtual-machine: ~/25SUB4508_LSP/25SUB4508_56133/Assignments/day35_CA_Software/Exercise_1/cpp-docker$
```

Step 4: Running the Docker Container

The Docker container is run by mapping the container port to the host port. The application starts successfully inside the container.

Screenshot Placeholder:

```
student@student-virtual-machine: ~/25SUB4508_LSP/25SUB4508_56133/Assignments/day35_CA_Software/Exercise_1/cpp-docker$ docker images
IMAGE                                ID                                DISK USAGE  CONTENT SIZE  EXTRA
cpp-microservice:latest             259af2fa30e9                     569MB       169MB         U
hello-world:latest                  d4aaab6242e0                     25.9kB      9.52kB        U
kernel-dev:latest                   cf7dcf92cae9                     797MB       222MB         U
kindest/node@sha256:7416a61b42b1662ca6ca89f02028ac133a309a2a30ba309614e8ec94d976dc5a 7416a61b42b1 1.45GB      445MB         U
student@student-virtual-machine: ~/25SUB4508_LSP/25SUB4508_56133/Assignments/day35_CA_Software/Exercise_1/cpp-docker$
```

Step 5: Testing the Microservice

The running microservice is tested using a browser or curl command to verify the output.

Screenshot Placeholder:

```
student@student-virtual-machine: ~/25SUB4508_LSP/25SUB4508_56133/Assignments/day35_CA_Software/Exercise_1/cpp-docker$ curl http://localhost:8080
Hello from C++ Docker container!
student@student-virtual-machine: ~/25SUB4508_LSP/25SUB4508_56133/Assignments/day35_CA_Software/Exercise_1/cpp-docker$
```

Conclusion

In this exercise, a C++ microservice was successfully containerized using Docker. The Docker image was built and executed, and the service was tested via an exposed port.

